HELIOS GUI User's Guide

Parmenion S. Mavrikakis

February 24, 2026

What this GUI does. The HELIOS GUI is a single-window front-end that runs the existing HELIOS project scripts exactly as command-line subprocesses:

- `run_sie.py` (subcommands: `prepare`, `solve`, `post`, `make-points`)
- `pytools/visualization.py` (cross sections, near-fields, R/T plotting)
- `dist_run_helios.py` (SSH-based distributed partitioning over nodes)
- Mesh viewing via `pytools/plot_mesh.py` helpers

The GUI does *not* re-implement solvers; it orchestrates them and displays logs, progress, and plots.

Contents

1 Requirements and Installation **4**
1.1 Python packages . . . 4
1.2 Project layout (expected) . . . 4

2 Starting the GUI **4**
2.1 Launch command . . . 4
2.2 The top bar . . . 4

3 Global Console, Progress, and Job Control **5**
3.1 Console log panel . . . 5
3.2 Progress bar behavior . . . 5
3.3 Bottom buttons . . . 5

4 Data Model: Inputs and Outputs **6**
4.1 Inputs: `sim_data/<sim>` . . . 6
4.2 Outputs: `sim_res/<sim>` . . . 6

5 Tab: Pre-processing **6**
5.1 Purpose . . . 6
5.2 Controls . . . 6
5.3 Materials: eps vs n,k . . . 7

6 Tab: Mesh Viewer **7**
6.1 Purpose . . . 7
6.2 Controls . . . 8
6.3 Interpreting colors and legend . . . 9

7 Tab: Solver **9**
7.1 Purpose . . . 9
7.2 Controls and meaning . . . 9
7.3 What you should see in outputs . . . 10

8 Tab: Generate Points **10**
8.1 Purpose . . . 10
8.2 Key concepts . . . 10
8.3 Controls and visibility rules . . . 11
8.4 Step selection (uniform vs per-axis) . . . 11
8.5 Output name behavior . . . 12
8.6 Examples . . . 12

9 Tab: Post-processing **12**
9.1 Purpose . . . 12
9.2 Controls . . . 13
9.3 Output organization . . . 13

10 Tab: Distributed Run **14**
10.1 Purpose . . . 14
10.2 Important prerequisites . . . 14
10.3 Controls . . . 15
10.4 What "distributed" means here . . . 15

11 Tab: Visualization **15**
11.1 Purpose . . . 15
11.2 Common controls . . . 15
11.3 Plot type: Cross sections . . . 16
11.4 Plot type: Fields (near-fields) . . . 17
11.5 Plot type: R/T . . . 18

12 Tab: Generate Table **19**
12.1 Purpose . . . 19
12.2 Controls . . . 19

13 Tab: Settings **20**
13.1 Purpose . . . 20
13.2 Controls . . . 20
13.3 Common environment variables to set . . . 21

14 Recommended Workflows **22**
14.1 Workflow A: Standard local run (most common) . . . 22
14.2 Workflow B: Distributed run . . . 22

15 Troubleshooting and FAQs **22**
15.1 "No simulation" warning . . . 22
15.2 Prepare fails: mesh not found or multiple meshes . . . 22
15.3 Solve fails: "No jobs" . . . 22
15.4 Post fails: missing points . . . 22
15.5 Visualization shows "No image produced" . . . 23
15.6 Distributed run: "runner not found" preflight . . . 23
15.7 Performance tips . . . 23

16 Appendix: GUI → CLI mapping (quick reference) 23
16.1 Pre-processing 23
16.2 Solve 23
16.3 Make points 23
16.4 Post 24
16.5 Distributed 24
16.6 Visualization 24
16.7 Make table 24

1 Requirements and Installation

1.1 Python packages

The GUI requires:

- **PySide6** (Qt GUI framework)
- **matplotlib** (embedded plotting)
- **numpy** (plotting and mesh helpers)
- **miepython** (used by `visualization.py` if available)

1.2 Project layout (expected)

The GUI is meant to run from a HELIOS repository root that contains, at minimum:

```
<root>/
  helios_gui.py
  run_sie.py
  dist_run_helios.py (for Distributed tab)
  apps/ (contains SIENano, SIENanoPP executables)
  pytools/ (visualization.py, plot_mesh.py, etc.)
  sim_data/<sim>/ (inputs: config.txt, mesh source file)
  sim_res/<sim>/ (outputs created by prepare/solve/post)
  logo.png (used in the GUI top bar)
```

Executables must exist and be executable. `run_sie.py` checks: `apps/SIENano` and `apps/SIENanoPP` exist and are executable. If they are missing or not executable, prepare/solve/-post will fail with a clear error.

2 Starting the GUI

2.1 Launch command

From the repository root:

```
python helios_gui.py --root <project_root>
```

If `--root` is omitted, the GUI uses the current directory.

2.2 The top bar

Across the top you will see:

- **Open...** toolbutton:
  - Click the main button: opens `sim_res/<sim>` in a file dialog.
  - Menu options:
    - `Open sim_res/<sim>`
    - `Open sim_data/<sim>`
- **Project root:** text field + **Browse...** button.
- **Simulation:** dropdown selector.
- ⟳ refresh button to rescan `sim_data/`.

- **New sim...** to create `sim_data/<name>` and `sim_res/<name>`.

Simulation list source. The Simulation dropdown is populated from subfolders under `sim_data/`. If a folder exists only under `sim_res/` but not under `sim_data/`, it will not appear until you create the matching `sim_data/<sim>` directory.

3 Global Console, Progress, and Job Control

3.1 Console log panel

All long-running operations executed through the GUI worker thread stream their stdout/stderr into the large console panel at the bottom. The GUI prepends commands with a `$` marker so you can copy/paste the exact CLI invocation.

3.2 Progress bar behavior

The GUI has a single global progress bar. It behaves as follows:

- For **Solve** runs, it parses lines from `run_sie.py`:

  ```
  [solve-progress] total N
  [solve-progress] done K
  ```

  Total progress is the sum of all `total` lines; ongoing progress increments by 1 for each `done`.

- For **Post-processing** runs, similarly:

  ```
  [post-progress] total N
  [post-progress] done K
  ```

- For all other tasks, the bar shows an indeterminate "busy" state until completion.
- **Distributed runs:** `dist_run_helios.py` emits `[dist-progress]` lines.

3.3 Bottom buttons

At the bottom right:

- **Clean console**: clears the console and resets the progress bar to `Ready`.
- **Cancel all**: terminates the currently running subprocess started by the GUI worker thread (prepare/solve/post/make-points/distributed).

Cancel behavior is POSIX-oriented. Cancellation is implemented by killing the subprocess *process group* (SIGTERM then SIGKILL). This is robust on Linux/macOS. On Windows, process-group semantics differ; cancellation may not reliably kill all children.

Visualization Render is not cancellable from "Cancel all". The Visualization tab runs `visualization.py` via a blocking call (`subprocess.run`) directly in the GUI thread. If plotting is slow, the UI can appear unresponsive, and Cancel will not interrupt it.

4 Data Model: Inputs and Outputs

4.1 Inputs: sim_data/<sim>

The solver workflow expects your simulation input folder:

```
sim_data/<sim>/
  config.txt
  <mesh_source>.mphtxt or <mesh_source>.msh
```

run_sie.py can auto-detect a single mesh source file under sim_data/<sim>/. If multiple mesh sources are present, you should specify the one you want in the GUI.

4.2 Outputs: sim_res/<sim>

A typical prepare step creates:

```
sim_res/<sim>/
  jobs/ job.1.xml, job.2.xml, ...
  mesh.mesh (converted mesh)
  out/ solver outputs
  points/ generated .pos point sets
  logs/ run logs
```

Solve outputs (from apps/SIENano) are typically written under sim_res/<sim>/out/ and include:

- *.sol solution files (e.g., sim<i>_inc_<j>.sol)
- out/csc/ gathered cross sections *.csc

Post-processing outputs (from apps/SIENanoPP) include field files, organized as:

```
sim_res/<sim>/out/fields/<points_name>/
  *.ein *.esc *.hin *.hsc
```

5 Tab: Pre-processing

5.1 Purpose

This tab wraps:

```
run_sie.py prepare <sim> --mode <0|1|2> [--meshfile ...] [--materials eps|nk] [--spline]
    [--overwrite]
```

It generates job.<N>.xml files and converts mesh inputs into mesh.mesh under sim_res/<sim>/.

5.2 Controls

Control	Meaning / Effect
Mode	Sets --mode passed to run_sie.py. Values: • 0: isolated scatterer(s) in homogeneous background • 1: periodic structure in homogeneous background • 2: isolated scatterer(s) in layered background
Mesh file (.mphtxt/.msh)	Optional explicit mesh source path. If left empty, run_sie.py auto-detects a single *.mphtxt or *.msh under sim_data/<sim>/.

Overwrite existing folder	If enabled, adds `--overwrite`. If disabled and `sim_res/<sim>` exists, the GUI will ask before overwriting outputs.
Materials columns	Selects `--materials eps` vs `--materials nk`.
Use spline interpolation	Adds `--spline` for cubic spline interpolation of tabulated materials.
Prepare simulation	Runs the command in the worker thread and streams output to console.

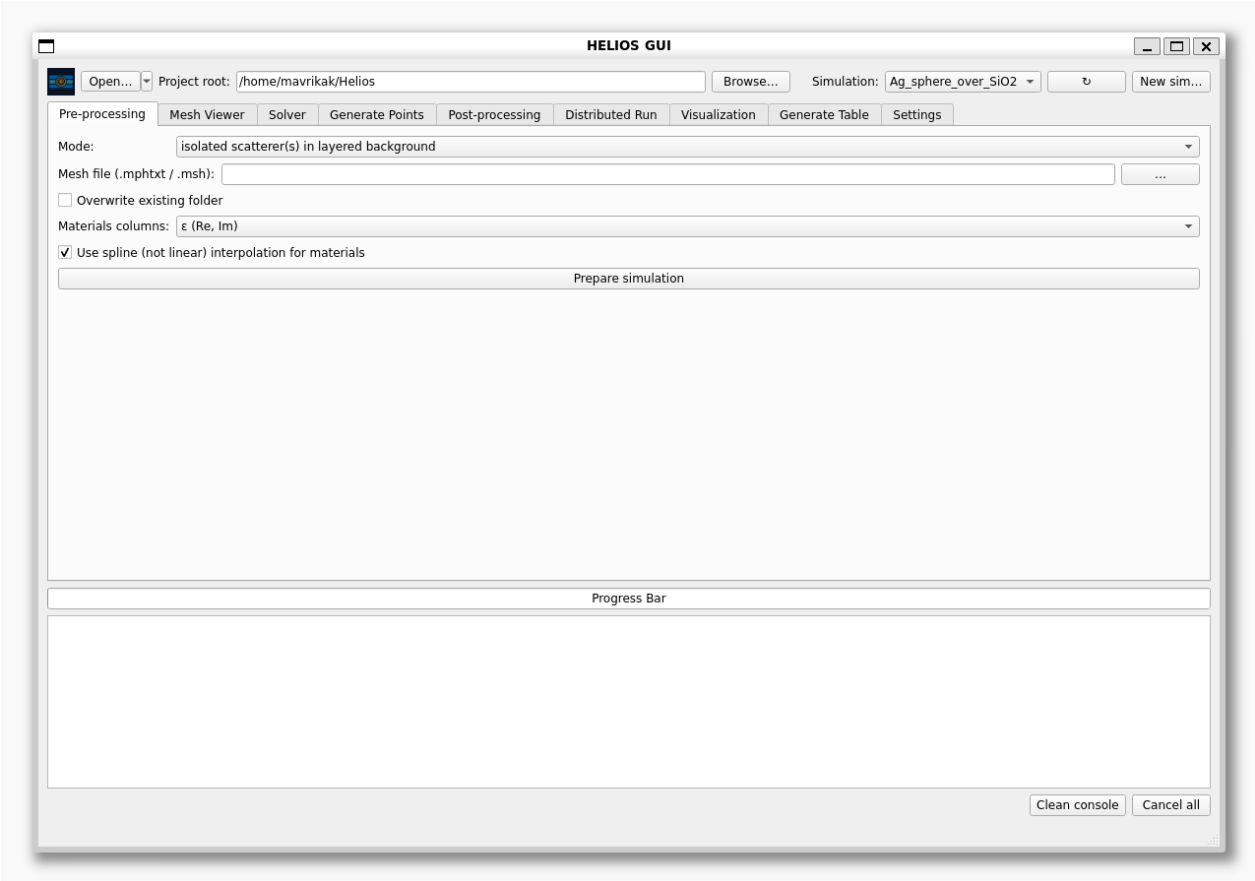

Figure 1: Pre-processing HELIOS GUI tab.

5.3 Materials: eps vs n,k

Material tables are used by the job generation logic:

- ε **tables** are interpreted as `[lambda[nm], Re(eps), Im(eps)]`.
- **n,k tables** are interpreted as `[lambda[nm], n, k]` and converted to ε.

This choice is forwarded to `jobwriter.py` via `run_sie.py`.

> If you have a shared materials repository, set `MATERIALS_DIR` in the Settings tab environment table (see §13).

6 Tab: Mesh Viewer

6.1 Purpose

This tab provides an integrated mesh preview (no external solver invocation). It automatically locates the converted mesh and interface planes:

- Mesh path: auto-detected `sim_res/<sim>/mesh.mesh` (or equivalent location resolved by helper).
- Interfaces: inferred from `sim_res/<sim>/config.txt` (layered backgrounds), when present.

6.2 Controls

Control	Meaning / Effect
Mode	
	• **3D**: full 3D mesh rendering (triangulated surface boundaries). • **2D: XY / XZ / YZ**: projected triangle edges onto the selected plane. • **1D: z-interfaces**: histogram of node z-distribution with interface lines.
Decimate	Integer downsampling factor for faces (reduces rendering load). A value of 1 draws all faces; 10 draws roughly 1/10.
Alpha	Transparency for the 3D mesh surface (useful for seeing interior lines/-planes).
Save to file + Filename	If enabled, saves a `.png` into `sim_res/<sim>/out/media/`. If the filename lacks `.png`, it is added. Absolute paths are also accepted.
Load / Render	Loads mesh + interfaces and renders the selected view.

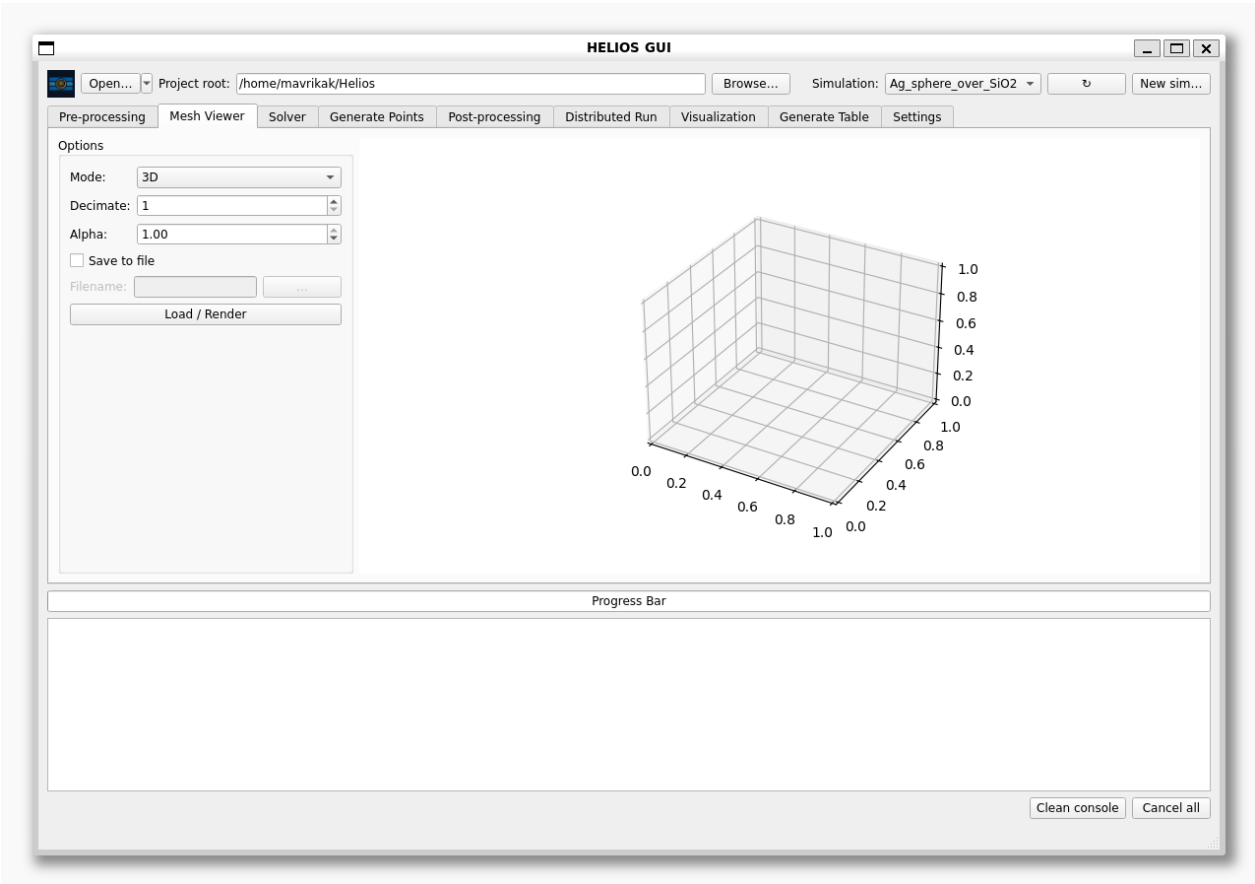

Figure 2: Mesh viewer HELIOS GUI tab.

6.3 Interpreting colors and legend

2D/3D views color faces by **domain boundary transitions** (front → back). The legend title indicates that the background domain is domain 0.

> If the Mesh Viewer fails, it usually indicates you have not run **Prepare** yet (so `mesh.mesh` does not exist), or your simulation outputs are in a non-standard location.

7 Tab: Solver

7.1 Purpose

This tab wraps:

```
run_sie.py solve <sim> --mode <0|1|2> -l <level> -th <threads> [-a]
  [-j <jobs>] or [--lambdas <csv>]
  [--etm <N>]
```

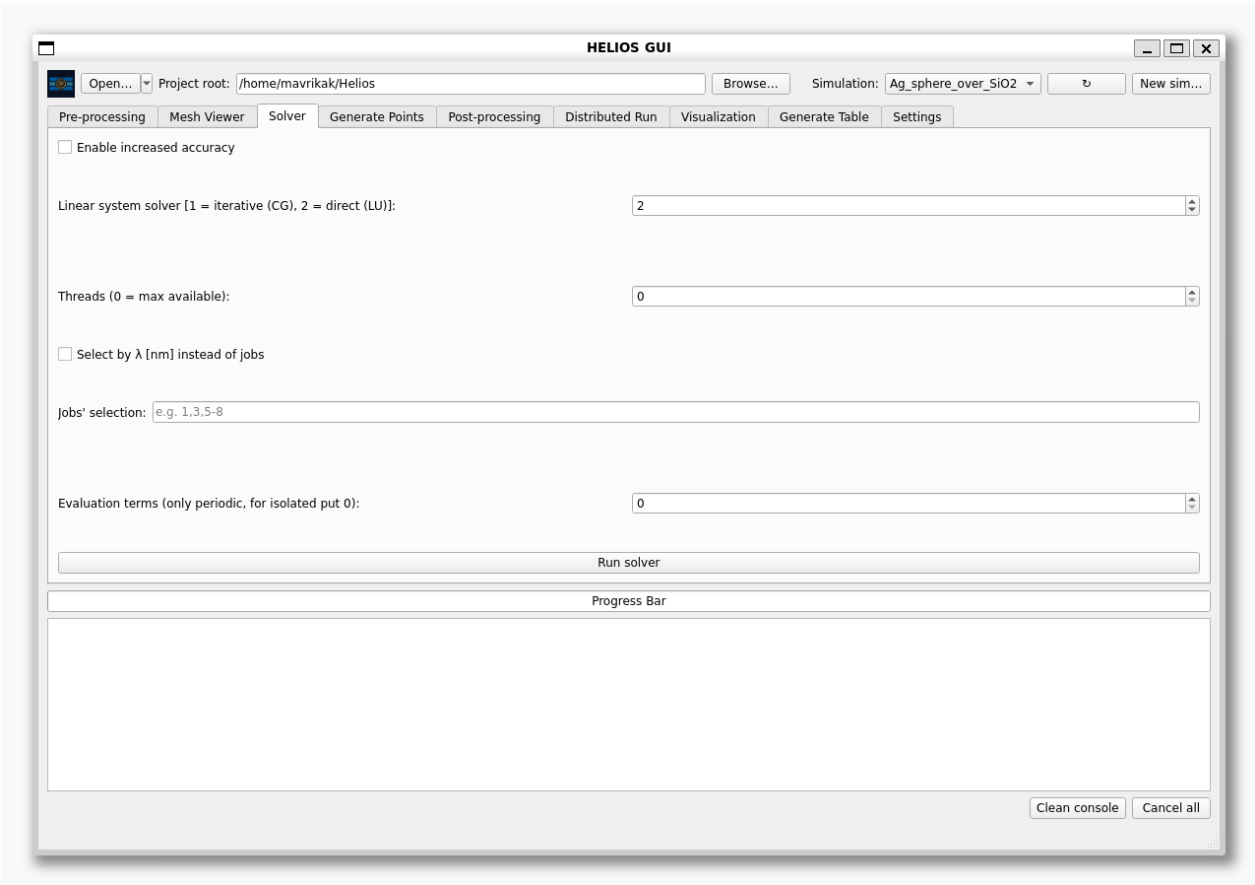

Figure 3: Solver HELIOS GUI tab.

7.2 Controls and meaning

Control	Meaning / Effect
Enable increased accuracy	Adds `-a`. This passes accuracy flags to `SIENano` through `run_sie.py`.

Linear system solver level	Forwarded as `-l`. Convention in the GUI: • 1 = iterative (CG) • 2 = direct (LU) (Other integers are accepted by the underlying solver, but typical use is 1 or 2.)
Threads	Forwarded as `-th`. `0` means "max available" (solver decides).
Select by jobs	"Jobs' selection" string like `1,3,5-8`. These are **1-based** job indices matching `job.<N>.xml`. Forwarded as `-j <spec>`.
Select by λ [nm]	If enabled, hides job selection and instead uses `-lambdas "<csv>"` (e.g. `500,650,750`). `run_sie.py` maps requested wavelengths to job indices (closest matches) using its wavelength metadata.
Evaluation terms (periodic)	Forwarded as `-etm N`. Only meaningful for periodic simulations (Mode 1). For isolated problems, set to 0 (or leave default).
Run solver	Launches the solver run for selected jobs.

7.3 What you should see in outputs

After a successful solve:

- Logs per job under `sim_res/<sim>/logs/` (e.g. `S1.txt`, `S2.txt`).
- Solution files `*.sol` under `sim_res/<sim>/out/`.
- Cross sections gathered into `sim_res/<sim>/out/csc/`.

> If you only want a small set of wavelengths, use either **jobs selection** or λ **selection** to avoid running unnecessary jobs.

8 Tab: Generate Points

8.1 Purpose

This tab wraps:

```
run_sie.py make-points <sim> <plane> --x xmin xmax --z zmin zmax ... --step...
  --x0/--y0/--z0 <comma_or_space_list> -o <output.pos>
```

It creates sampling point sets (`.pos`) to be used by post-processing and field visualization.

8.2 Key concepts

- The GUI generates **plane grids**: `xz`, `xy`, or `yz`.
- Each plane requires a list of **fixed-coordinate values**:
  - `xz` plane **requires** `y0` (one or more values)
  - `xy` plane **requires** `z0` (one or more values)
  - `yz` plane **requires** `x0` (one or more values)
- The output file is plain text: one `x y z` triplet per line, no header.

8.3 Controls and visibility rules

The GUI hides/shows coordinate fields depending on the chosen plane:

- Plane **xz**: shows `xmin,xmax,zmin,zmax,y0`.
- Plane **xy**: shows `xmin,xmax,ymin,ymax,z0`.
- Plane **yz**: shows `ymin,ymax,zmin,zmax,x0`.

Step controls and output name are always visible.

8.4 Step selection (uniform vs per-axis)

The GUI enforces an exclusive rule:

- If **step (uniform)** is filled, `stepx/stepy/stepz` are disabled.
- If any **axis step** is filled, `step (uniform)` is disabled.

At least one step must be provided for the relevant axes. `run_sie.py make-points` will error if required axis steps are missing. Example: for `xz` you must provide `step` or both `stepx` and `stepz`.

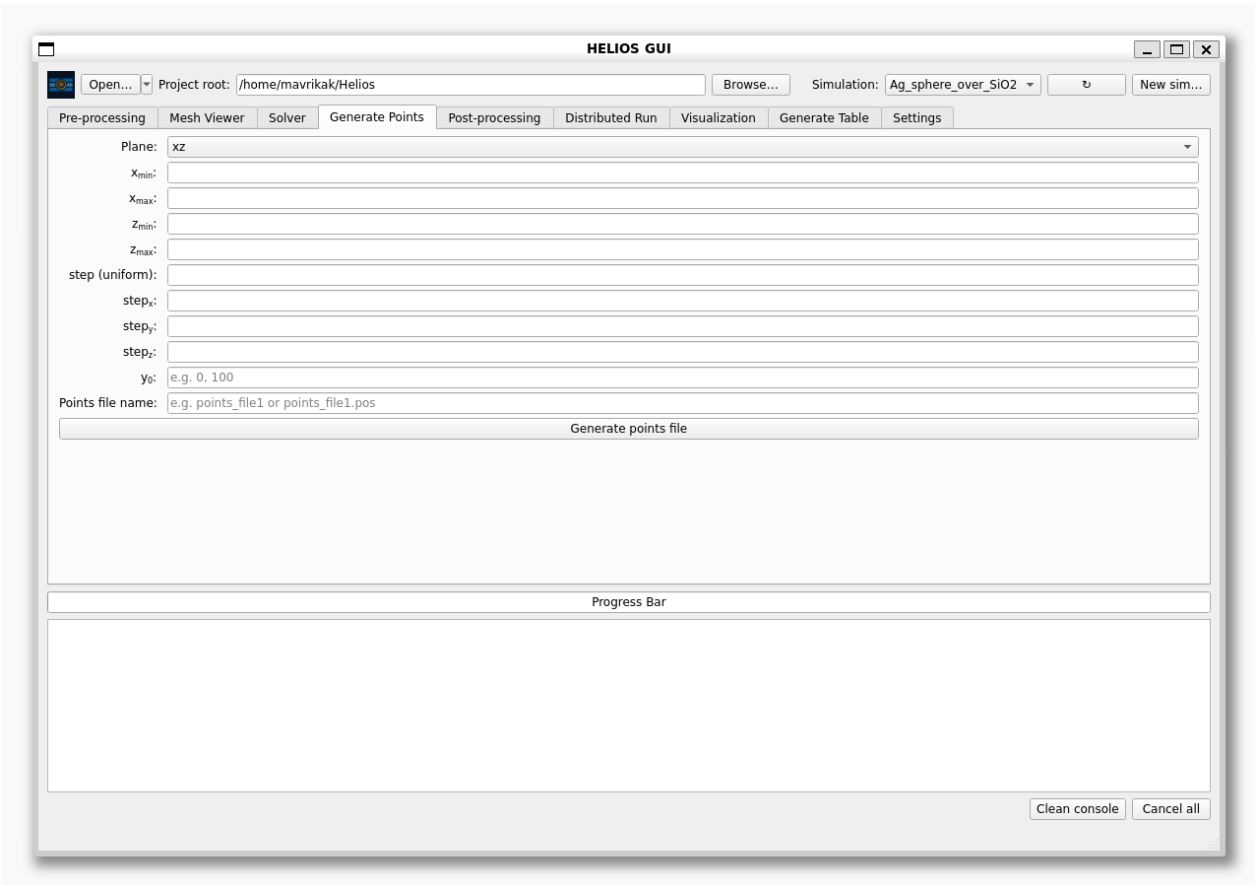

Figure 4: Generate points HELIOS GUI tab.

8.5 Output name behavior

- If the output name does not end with `.pos`, the GUI adds `.pos`.
- If you provide only a bare filename (no path), the GUI saves it under:

  `sim_res/<sim>/points/<name>.pos`

- If you provide an absolute path, it is used as-is.

8.6 Examples

Example A: xz plane at multiple y positions

- Plane: `xz`
- $x_{\min} = -500$, $x_{\max} = 500$
- $z_{\min} = -300$, $z_{\max} = 300$
- $y_0 = -200, 0, 200$
- step (uniform): `10`
- Output: `points_xz_multi`

This produces one combined points file containing three xz grids appended together.

Example B: xy plane at a single z slice with anisotropic steps

- Plane: `xy`
- $x_{\min} = -200$, $x_{\max} = 200$
- $y_{\min} = -200$, $y_{\max} = 200$
- $z_0 = 50$
- `stepx`=5, `stepy`=10
- Output: `plane_xy_z50.pos`

9 Tab: Post-processing

9.1 Purpose

This tab wraps:

```
run_sie.py post <sim> -th <threads> [-a] [--etm <N>]
  [-j <jobs>] or [--lambdas <csv>]
  -p <points1.pos> <points2.pos> ...
```

It runs `SIENanoPP` for each selected job and each provided points set, producing incident and scattered field files (`.ein/.esc/.hin/.hsc`).

9.2 Controls

Control	Meaning / Effect
Enable increased accuracy	Adds `-a` to the post-processing calls.
Threads	Adds `-th`. `0` means "max available" per solver.
Evaluation terms	Adds `-etm` (for periodic).
Select by jobs / by λ	Same logic as Solver tab, but applied to post-processing job selection.
Points file name	One or more point-set names or paths (space-separated). The GUI tries to resolve each token as: • absolute path (if exists), else • `sim_res/<sim>/points/<token>` or `<token>.pos`, else • falls back to `sim_res/<sim>/points/<token>.pos`.
Run post-processing	Launches the post-processing in the worker thread.

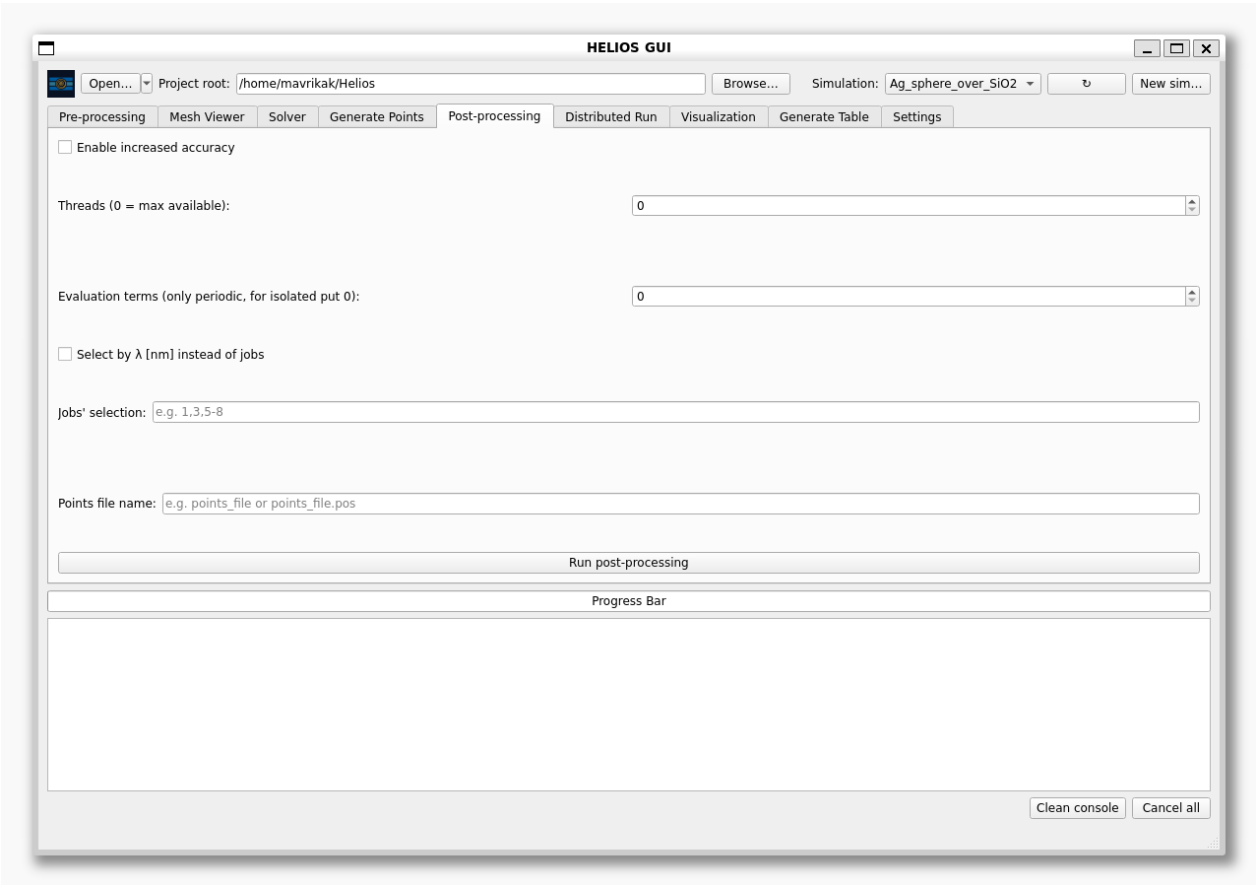

Figure 5: Post-processing HELIOS GUI tab.

9.3 Output organization

Field files are moved into:

```
sim_res/<sim>/out/fields/<points_name>/
```

where `points_name` is typically the `.pos` basename.

You can post-process multiple point sets in one run by entering multiple names separated by spaces. This will execute one SIENanoPP call per (job × point-set).

10 Tab: Distributed Run

10.1 Purpose

This tab runs distributed solve or post-processing across multiple nodes via SSH using:

```
dist_run_helios.py --mode <0|1|2> -s <sim> --root <root> -R <run_sie.py> -N <nodes_csv> <
    subcmd> <args...>
```

It partitions the job indices into contiguous chunks and runs `run_sie.py` remotely on each node with `-j` set to that slice.

10.2 Important prerequisites

You need passwordless SSH and consistent paths. Distributed mode assumes:

- Nodes are reachable by SSH without interactive password prompts.
- The project root exists on each node at the same path (or you provide `-root` that matches).
- `run_sie.py` exists on each node at the path you configured.

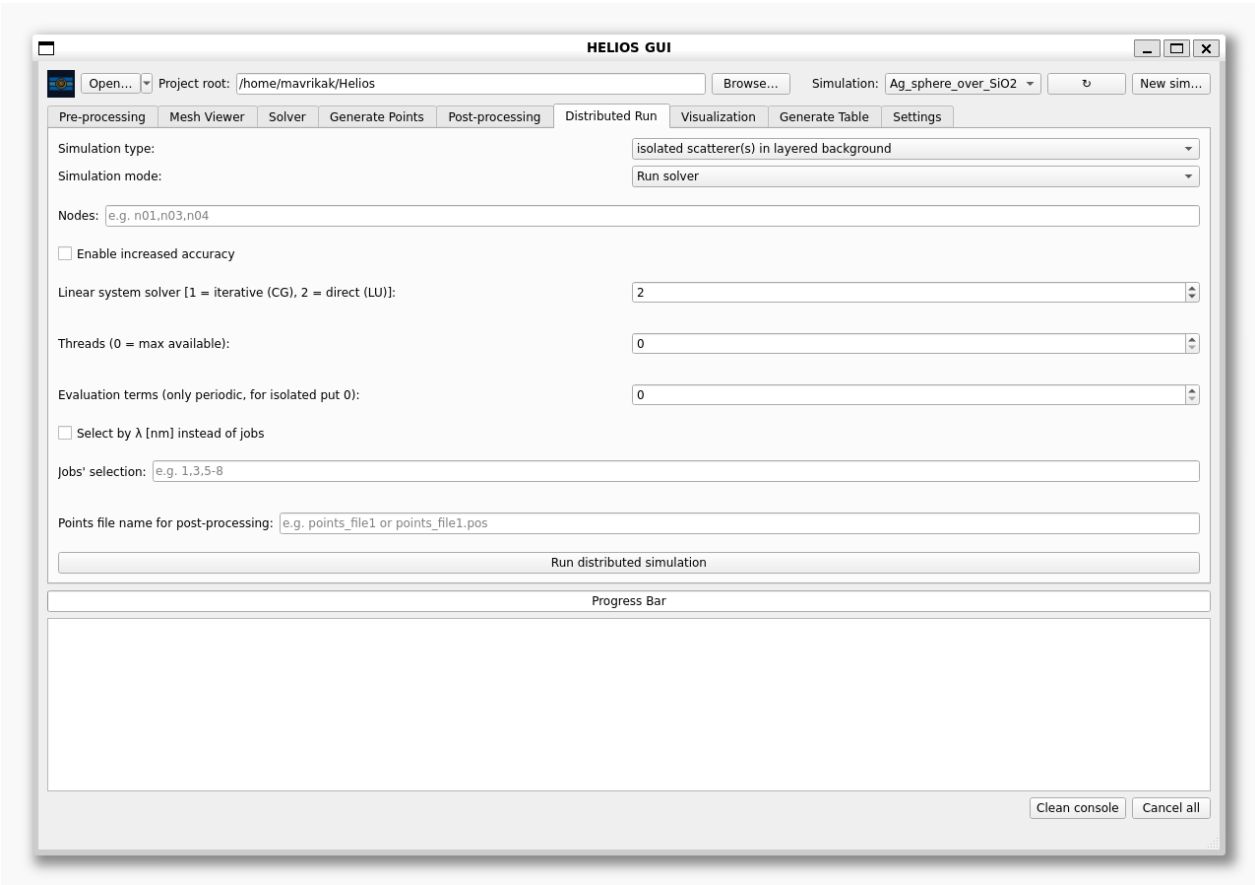

Figure 6: Distributed run HELIOS GUI tab.

10.3 Controls

Control	Meaning / Effect
Simulation type	Maps to `--mode 0|1|2` for distributed runner.
Simulation mode	Choose **Run solver** or **Run post-processing**.
Nodes	Comma-separated node list (e.g. `n01,n03,n04`). Forwarded as `-N`.
Enable increased accuracy	Adds `-a`.
Linear system solver level	For solver mode only; forwarded as `solve -l <level>`.
Threads	Forwarded as `-th <threads>` to remote `run_sie.py`.
Evaluation terms	For periodic (mode 1) forwarded as `-etm <N>` to remote `run_sie.py`.
Jobs selection / λ selection	Same selection logic as local runs; distributed runner removes any user `-j` and adds a different `-j` per node slice.
Points file name (post)	Required when running distributed post-processing. Resolved similarly to local post.
Run distributed simulation	Launches `dist_run_helios.py` in the GUI worker thread.

10.4 What "distributed" means here

`dist_run_helios.py`:

- Reads the list of available job indices from `sim_res/<sim>/jobs/job.*.xml`.
- Applies your selection (jobs or λ mapping) to get a final "desired" list.
- Partitions that list into contiguous chunks across nodes.
- On each node, runs `python3 run_sie.py solve/post ... -j <chunk>`.

Progress bar note. The distributed runner prints `[dist-progress] total` and `[dist-progress] done` lines. Use the console log to monitor slice completion.

11 Tab: Visualization

11.1 Purpose

This tab creates plots by calling `pytools/visualization.py` and then displays the generated PNG inside the GUI. It supports three plot types:

- **Cross sections**: SCS/ACS/EXT vs. wavelength (optionally multiple simulations)
- **Fields**: near-field maps from post-processed field files
- **R/T**: reflectance/transmittance vs. wavelength (for appropriate simulations)

11.2 Common controls

- **Subjob**: selects which subjob index to plot (forwarded as `--subjob`).
- **Save to file**:
  - If enabled and a filename is provided, the plot is saved to: `sim_res/<sim>/out/media/<name>.png` (unless you give an absolute path).
  - If disabled, the plot is rendered to a temporary file `<root>/__tmp_vis.png` and deleted after display.
- **Render**: runs `visualization.py` and updates the right-hand image panel.

11.3 Plot type: Cross sections

When **Cross sections** is selected, the GUI builds:

```
python visualization.py <sim1> <sim2> ... --root sim_res --plot-csc
  --subjob <N>
  --csc scs --csc acs --csc ext (based on your series selection)
  [--mie-radius R --mie-material ... --mie-background ... --materials-root materials --mie
      -spline]
  --save <png>
```

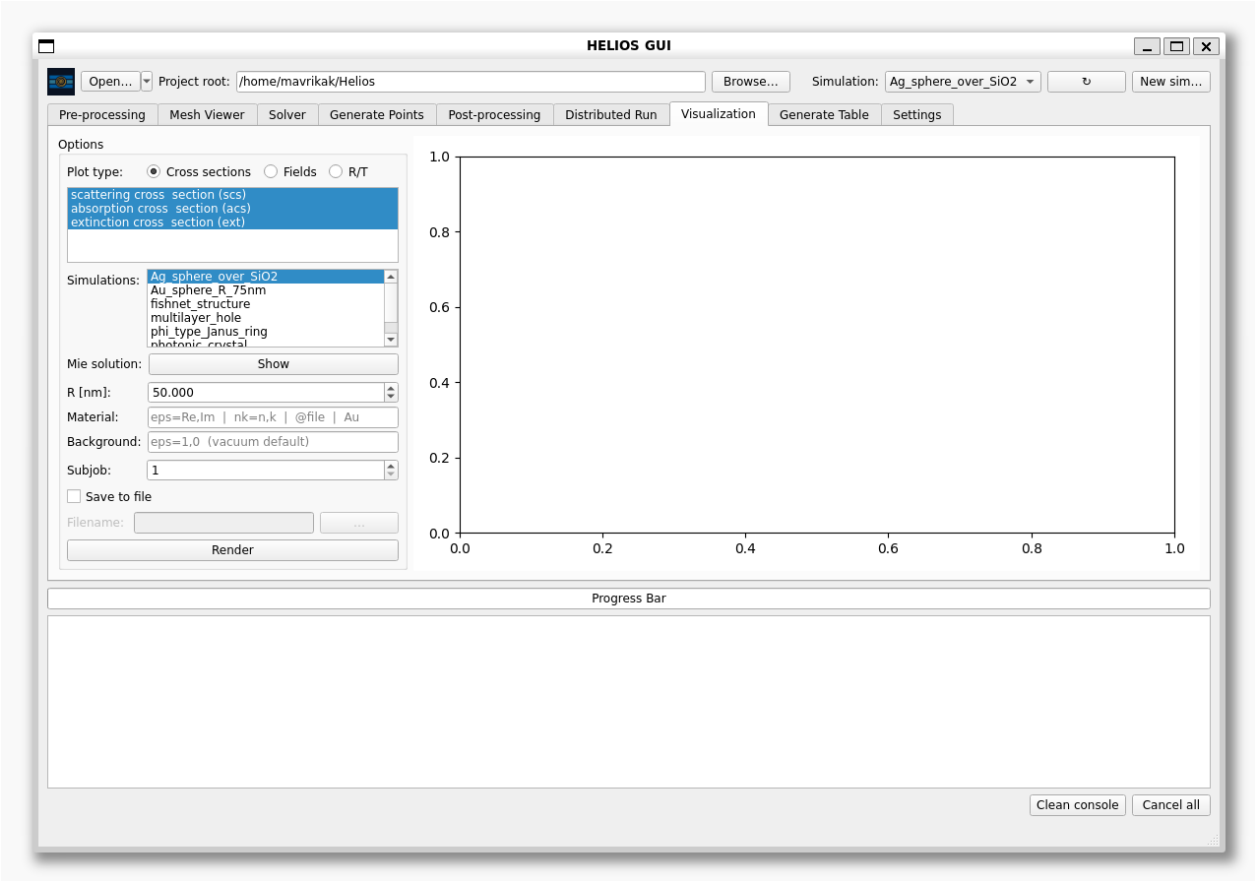

Figure 7: Visualization HELIOS GUI tab for cross sections.

Series selection A multi-select list lets you choose:

- scattering cross section (scs)
- absorption cross section (acs)
- extinction cross section (ext)

Multi-simulation overlay A multi-select list of simulations (from `sim_data/`) allows overlaying curves from several simulations in one plot.

Mie solution overlay (optional) There is a **Mie solution** block:

- Enable via the **Show/Hide** toggle.

- Provide:
  - **R [nm]** (radius)
  - **Material** (supports formats like `eps=Re,Im`, `nk=n,k`, or file reference
  - **Background** (default `eps=1,0`)

> **Mie requires miepython.** If `miepython` is not installed, `visualization.py` may not be able to compute Mie curves even if the GUI provides the options. Install `miepython` to enable this feature.

11.4 Plot type: Fields (near-fields)

When **Fields** is selected, the GUI builds:

```
python visualization.py <sim> --root sim_res --points <points.pos>
  --field e|h
  --part inc|sc|tot
  --quantity total|x|y|z
  --function intensity|magnitude|real|imag|phase
  --mode 2d|3d|1d
  --subjob <N>
  [--log]
  [--line "x=0" (for 1d mode)]
  [--lambda <nm>]
  [--hlines=z1,z2,... (if Interfaces enabled and interfaces found)]
  --save <png>
```

Inputs

- **Points name**: name or path to a `.pos` file. If you omit `.pos`, it is appended.
- Field files must exist from post-processing under: `sim_res/<sim>/out/fields/<points_name>/`.

Field and part

- **Field type**: `e` or `h`.
- **Field part**: `inc` (incident), `sc` (scattered), `tot` (total).

Quantity and function

- **Quantity**: `total` or a Cartesian component (`x,y,z`).
- **Function**:
  - `intensity` (commonly $|E|^2$-like)
  - `magnitude` (commonly $|E|$-like)
  - `real`, `imag`, `phase`

Dimension mode

- **2d**: heatmap on a rectilinear grid (best for points generated by `make-points`)
- **3d**: scatter plot fallback (best for irregular point clouds)
- **1d**: line cut through the plane (requires `1D line` selector)

1D line selector The **1D line** field accepts expressions like:

```
x=0
y=50
z=100
```

This is forwarded to `visualization.py --line`. Use it only when **Plot dimension** is `1d`.

Interfaces and logscale

- **Interfaces (Show/Hide)**: when enabled, the GUI tries to read interface z-values from `sim_res/<sim>/config.txt` and forwards them as `--hlines=z1,z2,...`.
- **Logscale (Enable/Disable)**: forwards `--log`.

λ selector If λ **[nm]** is set > 0, it is forwarded as `--lambda <nm>` and the plotting script will choose the closest available wavelength output.

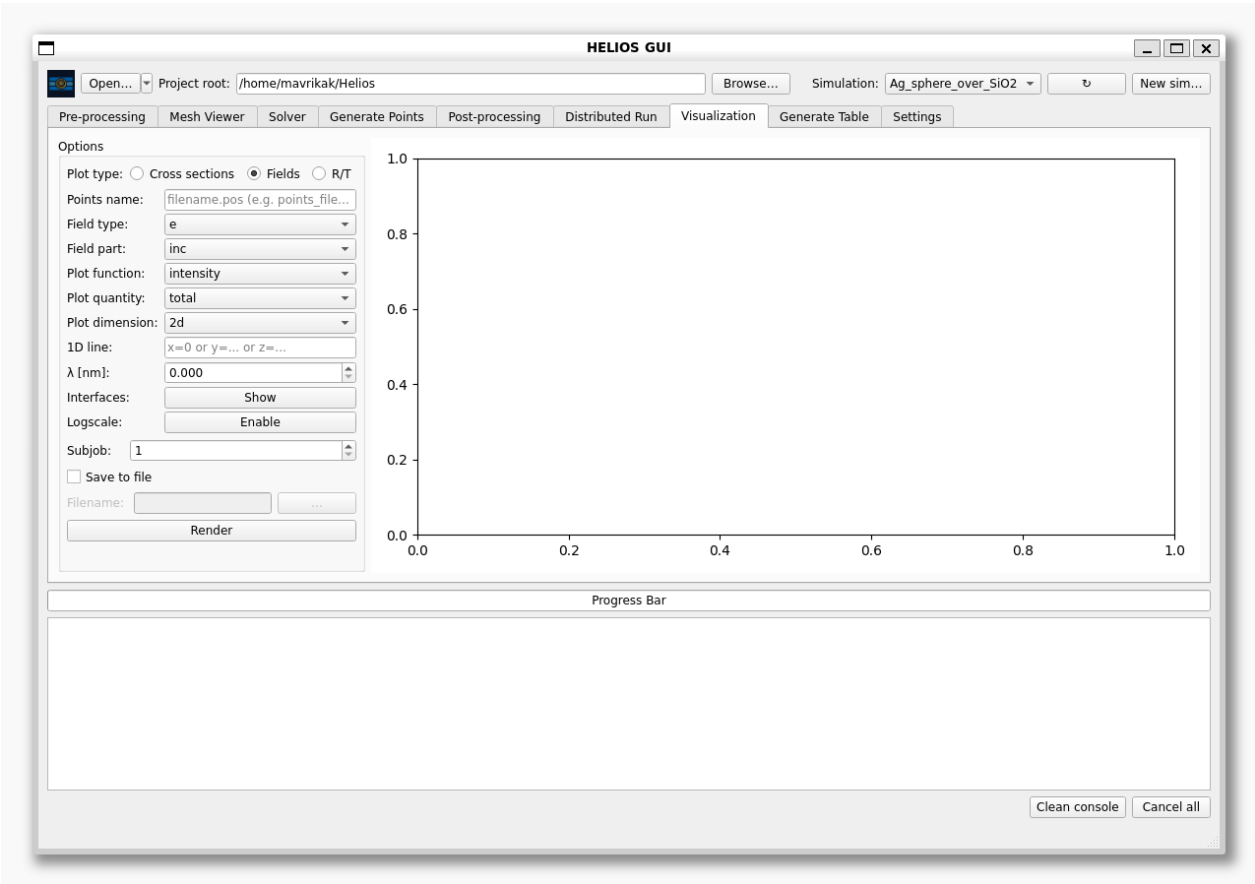

Figure 8: Visualization HELIOS GUI tab for fields.

11.5 Plot type: R/T

When **R/T** is selected, the GUI builds:

```
python visualization.py <sim> --root sim_res --plot-rt
  [--points <points.pos>]
  --subjob <N>
  --rt-prop auto|+1|-1 (based on Propagation selection)
  --save <png>
```

Propagation

- **auto**: let the script infer propagation direction.
- **+z**: forwarded as `-rt-prop +1`.
- **-z**: forwarded as `-rt-prop -1`.

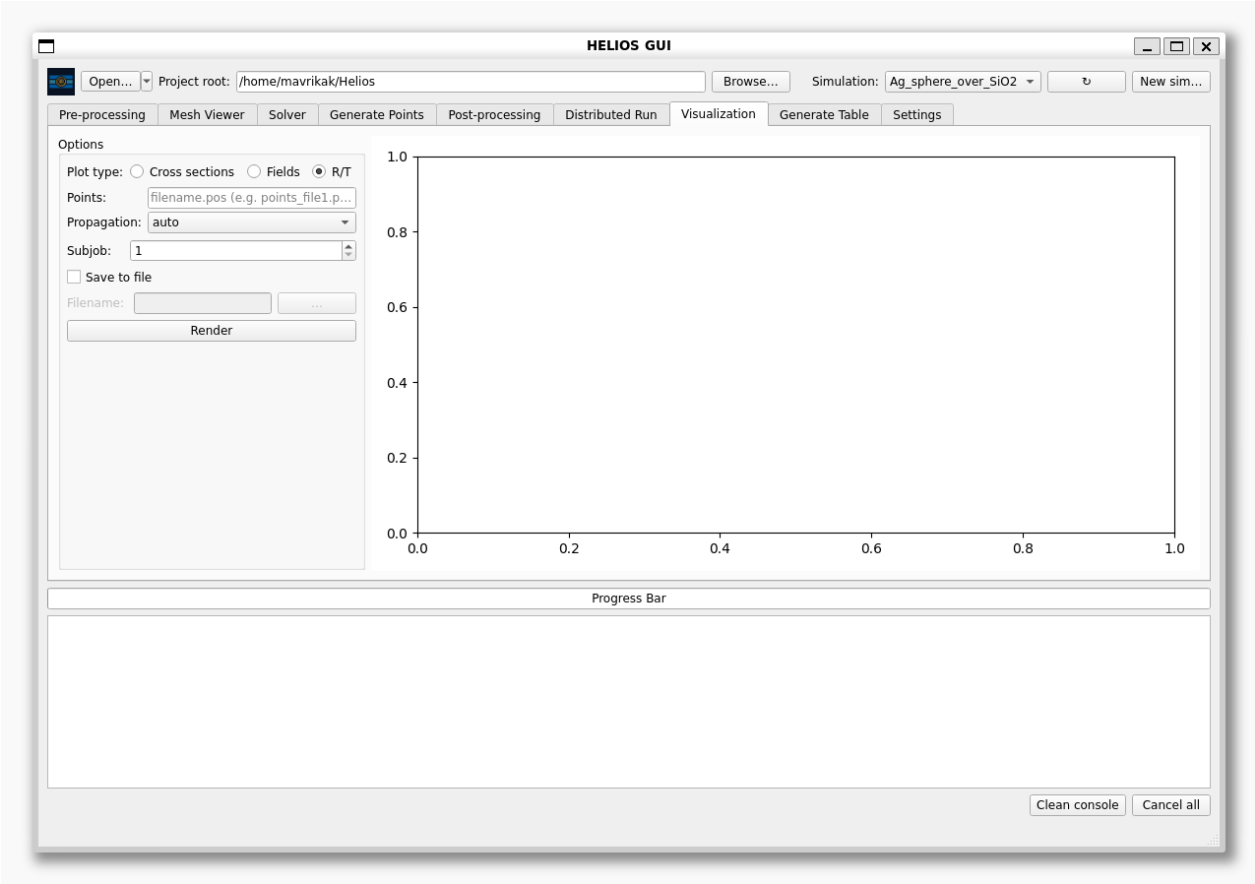

Figure 9: Visualization HELIOS GUI tab for reflectance and transmittance.

12 Tab: Generate Table

12.1 Purpose

This tab wraps:

```
run_sie.py make-table <filename> [--text] --maxRe <val> --maxIm <val> --incRe <val> --
    incIm <val>
```

It runs the `SIETable` application to generate a periodic lookup table (typically `erfc.bin`) and automatically moves it to the `third_party/` directory so it can be found by periodic jobs.

12.2 Controls

Control	Meaning / Effect
Output filename	Name of the output file (defaults to `erfc.bin`).

Save as text (not binary)	If enabled, adds `--text` to write a plain text file instead of a binary.
Max Re / Max Im	The maximum real and imaginary values for the table grid.
Inc Re / Inc Im	The increment (step size) for the real and imaginary axes.
Generate Table	Launches the table generation process in the background worker thread.

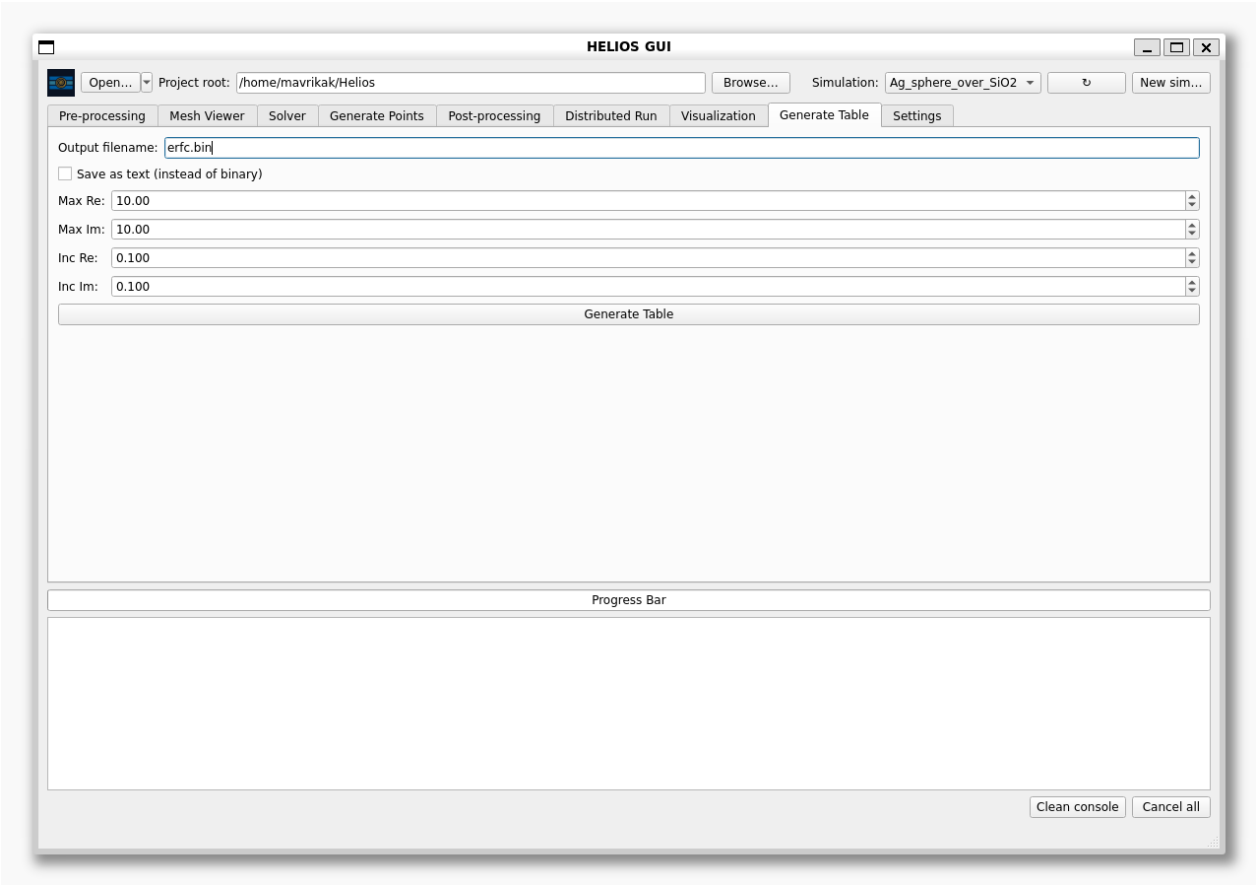

Figure 10: Generate table HELIOS GUI tab.

13 Tab: Settings

13.1 Purpose

This tab controls:

- Which Python interpreter is used to run subprocesses.
- The paths to `run_sie.py`, `visualization.py`, and `dist_run_helios.py`.
- Environment variables injected into the worker-thread runner jobs.

13.2 Controls

Field	Meaning / Effect
Python	Interpreter used for `run_sie.py` and `dist_run_helios.py` invocations in the worker runner. If empty, the GUI falls back to the current Python executable.

run_sie.py	Script path used by Pre-processing / Solver / Generate Points / Post-processing.
visualization.py	Script path used by Visualization tab.
Distributed runner	Script path used by Distributed Run tab (dist_run_helios.py).
Add environment variable	Adds key/value pairs to the environment of subprocesses launched through the GUI worker runner.

Environment variables do not automatically apply to Visualization. The Visualization tab runs `visualization.py` with a direct `subprocess.run` call and does not merge the Settings environment table into that process environment. If you need environment variables for visualization, set them in your shell before launching the GUI, or modify the code accordingly.

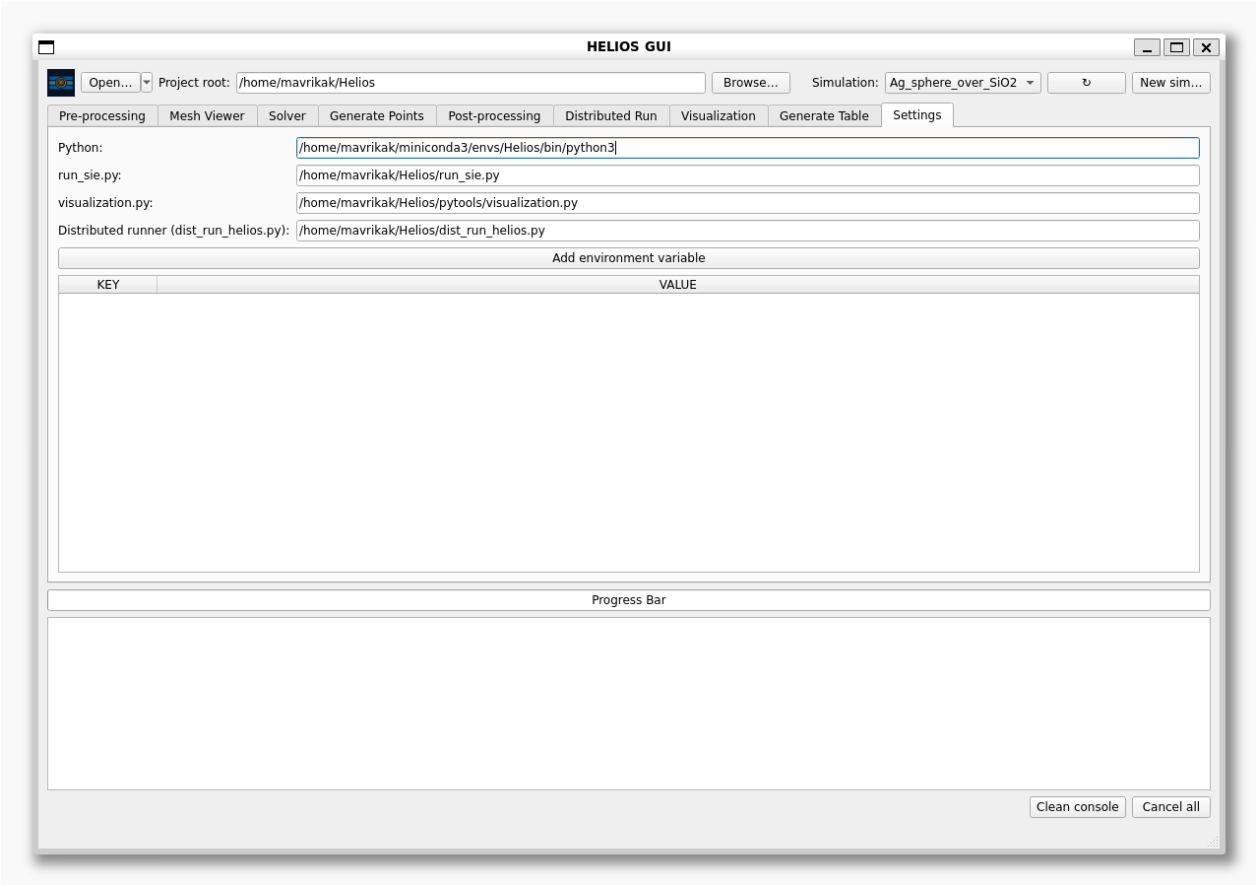

Figure 11: Settings HELIOS GUI tab.

13.3 Common environment variables to set

Depending on your setup, these can be useful:

- `MATERIALS_DIR`: override where materials tables are read from.
- `ERFC_TABLE`: default path to `erfc.bin` for periodic runs (unless `--table` is explicitly provided).
- `NO_BAR=1`: disables tqdm-like progress bars in CLI contexts.
- `SIE_NO_SPINNER=1`: disables lightweight spinners/prefix updates in CLI logs.

14 Recommended Workflows

14.1 Workflow A: Standard local run (most common)

1. **Create/select simulation**: choose a `sim_data/<sim>` folder (or click **New sim...**).
2. **Populate inputs** under `sim_data/<sim>/`: `config.txt` and a single mesh source (`.mphtxt` or `.msh`).
3. **Pre-processing** tab: select Mode, optionally select mesh file, then click **Prepare simulation**.
4. **(Optional) Mesh Viewer**: click **Load / Render** to inspect mesh quality and interface planes.
5. **Solver** tab: choose solver level, threads, job or λ selection; click **Run solver**.
6. **Generate Points** tab: generate one or more `.pos` plane files.
7. **Post-processing** tab: reference the points file(s) and click **Run post-processing**.
8. **Visualization** tab: choose **Fields** and render near-field plots, or **Cross sections**.

14.2 Workflow B: Distributed run

1. Perform **Prepare** locally first (recommended), ensuring jobs exist under `sim_res/<sim>/jobs/`.
2. Configure **Distributed runner** path in Settings.
3. In **Distributed Run** tab:
   - Fill **Nodes** with SSH-reachable hostnames.
   - Choose **Run solver** or **Run post-processing**.
   - Choose selection criteria (jobs or λ list).
   - Provide points for post-processing if needed.
   - Click **Run distributed simulation**.
4. Monitor progress in the console; each node slice is printed as `[dispatch] node -> joblist`.

15 Troubleshooting and FAQs

15.1 "No simulation" warning

Cause: no `sim_data/<sim>` folder is selected or the dropdown is empty.
Fix: click **New sim...** or create `sim_data/<sim>` manually, then press ⟳.

15.2 Prepare fails: mesh not found or multiple meshes

Cause: `sim_data/<sim>` contains no mesh, or contains multiple `*.msh/*.mphtxt`.
Fix: provide a single mesh source, or explicitly select one in the Pre-processing tab mesh field.

15.3 Solve fails: "No jobs"

Cause: `sim_res/<sim>/jobs/` is missing or empty.
Fix: run **Prepare simulation** first and confirm that `job.<N>.xml` files exist.

15.4 Post fails: missing points

Cause: Points file token cannot be resolved, or you did not generate points.
Fix: run **Generate points file** and then enter the correct points name (or full path).

15.5 Visualization shows "No image produced"

Causes:

- The plot script errored; check console lines `[visualization.py exit X]` and preceding output.
- The required input files are missing (e.g. no `out/csc/*.csc` for cross sections, no `out/fields/...` for fields).
- The points name does not match the available fields directory.

Fix: confirm the output folders exist; re-run solve/post; verify `points.pos` name matches the post-processing output folder.

15.6 Distributed run: "runner not found" preflight

Cause: remote nodes cannot find `run_sie.py` at the configured path relative to root.
Fix: ensure `--root` points to the same absolute path on each node, and that `-R` points to a valid `run_sie.py` path.

15.7 Performance tips

- Use **Mesh Viewer decimate** to speed up rendering large meshes.
- Use **job selection** or λ **selection** to run only what you need.
- Use a reasonable points grid step; very fine steps create huge field files and slow plots.

16 Appendix: GUI → CLI mapping (quick reference)

16.1 Pre-processing

```
python <run_sie.py> prepare <sim> --mode <0|1|2>
  [--meshfile <path>]
  --materials eps|nk
  [--spline]
  [--overwrite]
```

16.2 Solve

```
python <run_sie.py> solve <sim> --mode <0|1|2> -l <level> -th <threads>
  [-a]
  [--lambdas "500,650,..."] or [-j "1,3,5-8"]
  [--etm <N>]
```

16.3 Make points

```
python <run_sie.py> make-points <sim> <xz|xy|yz>
  --x <xmin> <xmax> (if relevant)
  --y <ymin> <ymax> (if relevant)
  --z <zmin> <zmax> (if relevant)
  --step <h> or per-axis: --stepx/--stepy/--stepz
  --x0/--y0/--z0 "comma,space,list" (required, depending on plane)
  -o <output.pos>
```

16.4 Post

```
python <run_sie.py> post <sim> -th <threads>
  [-a]
  [--etm <N>]
  [--lambdas "..."] or [-j "..."]
  -p <points1.pos> <points2.pos> ...
```

16.5 Distributed

```
python <dist_run_helios.py> --mode <0|1|2> -s <sim> --root <root> -R <run_sie.py>
  -N nodeA,nodeB,...
  solve -l <level> ... OR post -p <points...> ...
```

16.6 Visualization

```
python <visualization.py> ... --save <output.png>
```

16.7 Make table

```
python <run_sie.py> make-table <filename>
  [--text]
  --maxRe <maxRe> --maxIm <maxIm>
  --incRe <incRe> --incIm <incIm>
```